

Programação Web II – Aula

2 Componentes React e JSX – Props e Estado

Projeto: Sistema de Gestão Escolar



Por que usar Componentes?

Imagine construir um site como montar com peças de LEGO: cada peça é independente, reutilizável e encaixa com as outras. Se precisar mudar o rodapé, você muda **em um lugar só** — e toda a aplicação é atualizada.

Componentes são funções que devolvem pedaços de tela, organizando o código, evitando repetição e facilitando o trabalho em equipe.

Reutilizável

Use o mesmo componente quantas vezes quiser

Fácil de Manter

Mude em um lugar, reflete em todo o sistema

Trabalho em Equipe

Cada dev cuida de uma peça do sistema



O que é um Componente React?

Um componente é uma **função JavaScript** que retorna JSX — ou seja, HTML dentro do JavaScript. Componentes sempre começam com **letra maiúscula** e podem ser usados quantas vezes forem necessários.

```
// Componente simples
function Cabecalho() {
  return (
    <header>
      <h1>Sistema de Gestão Escolar</h1>
    </header>
  );
}

// Usando o componente (quantas vezes quiser!)
<Cabecalho />
<Cabecalho />
```

 A letra maiúscula no nome é obrigatória — é assim que o React diferencia componentes de tags HTML comuns.

JSX na Prática

JSX *parece* HTML, mas é JavaScript. Por isso, algumas regras mudam:

class → className

Palavra reservada no JS

Atributos em camelCase

onclick → onClick

Expressões entre {chaves}

Para inserir valores JavaScript no JSX

```
// JSX com expressões JS
const nome = "Gestão Escolar";

return (
  <div className="cabecalho">
    <h1>{nome}</h1>
    <p>{2 + 2} disciplinas</p>
  </div>
);

// O Vite converte tudo para JS puro!
```

Props – Dados de Entrada

Props são como os **ingredientes de uma receita**: o componente recebe dados de fora e os utiliza para montar a tela. O fluxo é sempre **de cima para baixo** — do componente pai para o filho.

1

Pai passa a prop

```
<Cabecalho titulo="Gestão Escolar" />
```

2

Filho recebe

```
function Cabecalho(props)
```

3

Filho exhibe

```
{props.titulo} na tela
```



Código: Cabecalho.jsx

Arquivo: frontend/src/componentes/Cabecalho.jsx

```
// 1. Definimos a função com props como parâmetro
function Cabecalho(props) {

  // 2. Retornamos JSX com os dados recebidos
  return (
    <header className="cabecalho">
      <h1>{props.titulo}</h1>      {/* título dinâmico */}
      <p>{props.subtitulo}</p>    {/* subtítulo dinâmico */}
    </header>
  );
}

// 3. Exportamos para poder usar em outros arquivos
export default Cabecalho;
```

📄 Com props diferentes, o mesmo componente gera resultados diferentes — sem duplicar nenhum código.



Estado com useState

Estado é um **dado que muda na tela sem recarregar a página** — como o placar de um jogo que atualiza em tempo real.

`useState` retorna dois valores: o dado atual e a função que o atualiza. Ao chamar a função, o React re-renderiza só o que mudou.

```
const [valor, setValor] =  
  useState(0);
```

```
// Ao clicar, o estado muda e a  
tela atualiza
```

```
<button onClick={() =>  
  setValor(valor + 1)}>
```

```
  Incrementar
```

```
</button>
```

Código: Contador.jsx

Arquivo: frontend/src/componentes/Contador.jsx

```
import { useState } from "react"; // 1. Importa o hook

function Contador() {
  // 2. Cria o estado: valor inicial = 0
  const [valor, setValor] = useState(0);

  return (
    <div>
      <p>Contagem: {valor}</p> {/* 3. Exibe o valor atual */}

      {/* 4. onClick chama setValor para atualizar o estado */}
      <button onClick={() => setValor(valor + 1)}>
        Incrementar
      </button>

      {/* 5. Zerar volta o estado para 0 */}
      <button onClick={() => setValor(0)}>Zerar</button>
    </div>
  );
}

export default Contador;
```

Código: App.jsx

O `App.jsx` é o componente raiz — ele importa e **combina** os outros componentes para montar a página completa.

```
import Cabecalho from "../componentes/Cabecalho"; // importa
import Contador from "../componentes/Contador"; // importa

function App() {
  return (
    <div>
      {/* Componente com props passadas */}
      <Cabecalho
        titulo="Sistema de Gestão Escolar"
        subtítulo="Bem-vindo!"
      />

      {/* Componente com estado próprio */}
      <Contador />
    </div>
  );
}

export default App;
```

 Cada componente cuida de si mesmo — o App só organiza quem aparece e em que ordem.

Erros Comuns em React

1

Esquecer o `export default`

O `import` falha silenciosamente — o componente simplesmente não aparece.

2

Usar `class` em vez de `className`

O React exibe um aviso no console e o estilo pode não funcionar.

3

Componente sem `return`

A tela fica em branco — cheque se há um `return` com JSX válido.

4

Esquecer de importar o componente

Erro no console: *"X is not defined"* — adicione o `import` no topo do arquivo.



Exercício e Desafio



Exercício

Crie o componente `Saudacao` que recebe a prop `nome` e exibe:

"Olá, {nome}!"

Teste usando `<Saudacao nome="Ana" />` no `App.jsx`.



Desafio Opcional

No componente `Contador`:

- O botão "Zerar" já foi implementado — certifique-se que funciona
- Limite o contador para não ultrapassar **10** (use um `if` dentro do `onClick`)



Próxima aula:
deixaremos o sistema
bonito com **Bootstrap!**